Daily Quant Digest: 2026-11-01

Foundations Restart, Revision Day 1 (Sunday). No new lessons today. We have climbed six steps in each track this week. Today we look back down the staircase: one picture per lesson, then fresh problems that make you use several lessons at once. If a problem feels hard, that is the point: mixing tools is exactly what interviews test.

Today’s topics

Track	Topic	Difficulty
Probability & Statistics	FR Revision: Lessons 1 to 6 (the room of worlds, events, counting I to III, conditional probability)	Easy-Medium
Derivatives	FR Revision: Lessons 1 to 6 (rates, discounting, markets, FX quotes, forwards, forward pricing) on one exporter’s hedge	Easy-Medium
Algorithms & DS	FR Revision: Lessons 1 to 6; variant problem: group anagrams and count anagram pairs (sorting + hash map + counting)	Easy-Medium
Music	The Kinks, “Waterloo Sunset” (1967)	

1. Probability & Statistics: The Week on One Page

> **Where we are.** Six lessons so far: what probability is, the language of events, three lessons of counting, and conditional probability. Every one of them rests on a single picture: **the room of possible worlds.** Today we walk through the room once more and then solve new problems with it.

1.1 The week in six pictures

Lesson	The picture to remember	The rule it gives
L1: what probability is	A **room of possible worlds**. Each outcome is one world.	Equally likely worlds: $P(A) = \frac{\#\text{worlds in } A}{\#\text{all worlds}}$
L2: events	**Guest lists**: "or" merges two lists, "and" keeps names on both, "not" is everyone else	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$; $P(\text{not } A) = 1 - P(A)$
L3: counting I	A **road with forks**: at each step, count the choices	Multiply the choices. n people in n chairs: $n!$ ways
L4: counting II	The **queue vs the bag**: a bag forgets order	$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$
L5: counting III	**Coins and fence posts**: n identical coins, $k-1$ posts to split them into k boxes	$\binom{n+k-1}{k-1}$ ways
L6: conditional probability	**News moves the walls**: you stand in a smaller room	$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$

Read the table as a staircase. L1 says "count worlds". L3 to L5 are tools for counting worlds when there are too many to list. L2 and L6 tell you **which** worlds to count: L2 combines events, L6 throws away worlds that disagree with the news.

1.2 One old idea revisited: count the opposite

There is not yet enough history for the usual "lesson from 2 to 6 weeks ago", so today we revisit the one trick that appeared in four of the six lessons: **the complement**.

The picture. You want to count the people at a party who are wearing at least one red item. That is a mess: one red item, two, three… It is much easier to count the people wearing **no** red at all and subtract from the total.

The rule. $P(\text{at least one}) = 1 - P(\text{none})$.

Where we used it. L2: numbers from 1 to 1,000 divisible by none of 2, 3, 5. L4: hands with at least one ace. L5: upper caps on boxes (count the bad shares and remove them). Today's interview question uses it again.

When to reach for it. Whenever you see the words "at least one", or a condition that splits into many messy cases while its opposite is a single clean case.

1.3 Practice problems (answers in 1.6)

1. **(L1 + L3)** Three traders each pick a whole number from 1 to 6 at random, independently. What is the probability that all three numbers are different?
2. **(L2)** How many whole numbers from 1 to 360 are divisible by 4, by 6, or by 9?

3. **(L4)** A desk has 10 traders: 6 FX and 4 rates. A random group of 4 is chosen for weekend cover. What is the probability it has exactly 2 FX traders?
4. **(L5 + L2)** 10 identical risk lots are shared among 4 accounts. Each account must get at least 1 lot and at most 4. How many ways are there?

1.4 Interview question: kings and two kinds of news

(A warm-up of the kind SIG and Optiver use. It mixes combinations, the complement and conditional probability. The trap is defining the small room carefully.)

You are dealt 5 cards from a well-shuffled 52-card deck.

a. What is the probability that your hand contains at least one king?

b. A friend peeks and says: "you have at least one king." Now what is the probability that you have at least two kings?

c. Instead the friend says: "you have the king of hearts." Now what is the probability that you have at least two kings?

d. Which of (b) and (c) is bigger, and why, in one sentence?

1.5 Solution

Step 0: the room. A hand is a bag of 5 cards (order does not matter), so by L4 the room has

$$\binom{52}{5} = 2{,}598{,}960 \text{ worlds, all equally likely.}$$

(a) At least one king: count the opposite. "No king" means all 5 cards come from the 48 non-kings:

$$\binom{48}{5} = 1{,}712{,}304.$$

So hands with at least one king: $2{,}598{,}960 - 1{,}712{,}304 = 886{,}656$, and

$$P(\text{at least one king}) = \frac{886{,}656}{2{,}598{,}960} \approx 0.341.$$

About one hand in three.

(b) The news "at least one king". The walls move in: the small room is the 886,656 hands from (a). Inside it we want "at least two kings". Count the opposite **inside the small room**: hands with **exactly one** king. Choose the king (4 ways) and 4 non-kings ($\binom{48}{4} = 194{,}580$ ways), by the multiplication rule of L3:

$$4 \times 194{,}580 = 778{,}320.$$

So hands with at least two kings: $886{,}656 - 778{,}320 = 108{,}336$, and

$$P(\text{at least two} \mid \text{at least one}) = \frac{108{,}336}{886{,}656} \approx 0.122.$$

(c) The news "you have the king of hearts". A different small room. Fix the king of hearts in the hand; the other 4 cards are any 4 of the remaining 51:

$$\binom{51}{4} = 249{,}900 \text{ hands.}$$

"Only one king" means those 4 cards avoid the other 3 kings, so they come from the 48 non-kings: $\binom{48}{4} = 194{,}580$. Hands with at least one more king: $249{,}900 - 194{,}580 = 55{,}320$, and

$$P(\text{at least two} \mid \text{king of hearts}) = \frac{55{,}320}{249{,}900} \approx 0.221.$$

(d) Why (c) is bigger. "At least one king" lets in all the many hands with exactly one king of any suit, which dilutes the room, while "the king of hearts" pins one king down and leaves 4 free cards that can still catch another king.

Interview trick for (c). Once the king of hearts is fixed, you just need "at least one of the 3 other kings among 4 cards drawn from 51". Count the opposite in your head: $\frac{48}{51} \cdot \frac{47}{50} \cdot \frac{46}{49} \cdot \frac{45}{48} \approx 0.779$, so the answer is about $1 - 0.779 = 0.221$.

Sanity checks.

- b. must be smaller than (c): the vaguer news lets in more one-king hands. It is (0.122 vs 0.221).
- b. must be bigger than the plain chance of at least two kings, $108{,}336/2{,}598{,}960 \approx 0.042$, because the news ruled out the zero-king hands. It is.
- This is yesterday's two-card puzzle (both aces given "at least one ace" vs "the ace of spades") in a bigger room. Same lesson: **the exact wording of the news decides the room.**

1.6 Answers to the practice problems

1. **5/9.** Road with forks (L3): the first trader has 6 choices, the second must avoid that number (5), the third avoids both (4): $6 \times 5 \times 4 = 120$ good worlds out of $6^3 = 216$. $120/216 = 5/9 \approx 0.556$.
2. **140.** Multiples of 4: $360/4 = 90$. Of 6: 60. Of 9: 40. "4 and 6" means multiples of 12: 30. "4 and 9" means 36: 10. "6 and 9" means 18 (not 54!): 20. All three means 36: 10. Inclusion-exclusion: $90 + 60 + 40 - 30 - 10 - 20 + 10 = 140$. The trap: the lowest common multiple of 6 and 9 is 18, not 6×9.
3. **3/7.** Bags (L4): choose 2 of 6 FX ($\binom{6}{2} = 15$) and 2 of 4 rates ($\binom{4}{2} = 6$): $15 \times 6 = 90$. All groups: $\binom{10}{4} = 210$. $90/210 = 3/7 \approx 0.43$.

4. **44.** Give each account 1 lot first; 6 lots remain for 4 accounts with no lower limit: $\binom{6+3}{3} = 84$. Now remove the bad shares, where some account ends up with 5 or more, i.e. gets 4 or more of the remaining 6. Pick that account (4 ways), give it 4 more, and share the last 2 freely: $\binom{2+3}{3} = 10$. Two accounts cannot both be bad (that needs 8 extra lots). So $84 - 4 \times 10 = 44$.

---

2. Derivatives: One Exporter, Six Lessons

> **Where we are.** Lessons 1 and 2 taught interest and discounting. Lesson 3 showed how a market quotes a bid and an ask. Lesson 4 taught FX quotes and pips. Lessons 5 and 6 built the forward contract and priced it by no-arbitrage. Today we follow **one company through one hedge** and use every lesson along the way.

2.1 The week in six pictures

Lesson	The picture to remember	The rule it gives
L1: interest rates	A **snowball** rolling downhill: interest earns interest	Simple $1 + rt$; compound $(1 + r/m)^{mt}$; continuous e^{rt}
L2: present value	**Future money is a foreign currency**: convert it to today's money with a rate	$DF = 1/(1 + rt)$ or e^{-rt}; $PV = DF \times$ cash
L3: markets	The **airport money-changer**: buys low (bid), sells high (ask)	You sell at the bid, buy at the ask; spread = ask minus bid
L4: FX quotes	A **price tag**: the base currency is on the shelf, the quote currency is in your wallet	EURUSD 1.1581 = 1 EUR costs 1.1581 USD; pip value = 0.0001 x base notional
L5: forwards	A **pre-order**: fix the price today, swap the goods later	Short forward payoff: $N(K - S_T)$
L6: forward pricing	**Two roads to the same village** must cost the same	$F = S \times \dfrac{1 + r_{\text{quote}}\, t}{1 + r_{\text{base}}\, t}$

2.2 The scene

A Hong Kong electronics exporter sells speakers to a German retailer. The retailer will pay **EUR 8 million in 182 days**. The exporter's costs and reporting are in US dollars (the HKD is pegged to USD, so think USD). The CFO worries: if the euro falls over the next six months, those 8 million euros will buy fewer dollars.

Her bank's FX desk shows these numbers:

Market data	Value
EURUSD spot	1.1580 / 1.1582 (mid 1.1581)
USD 6-month rate	3.80% (simple, ACT/360)
EUR 6-month rate	2.10% (simple, ACT/360)
Days	182, so $t = 182/360 = 0.50556$

Try each step yourself before reading the answer.

2.3 Step 1 (L1): how big is the snowball?

Question. How much interest does USD 1 million earn over these 182 days at 3.80%? What is the same rate written as continuous compounding?

Answer. Simple interest: $1{,}000{,}000 \times 0.038 \times 0.50556 = \text{USD } 19{,}211$. So the growth factor is $1 + rt = 1.019211$.

The continuous rate r_c must give the same growth over the same time: $e^{r_c t} = 1.019211$. Take the natural log of both sides: $r_c = \ln(1.019211)/0.50556 = 3.764\%$. It is a bit **smaller** than 3.80% because continuous compounding lets interest earn interest, so it needs a lower headline rate to reach the same place.

2.4 Step 2 (L2): what is a future dollar worth today?

Question. What is the 182-day USD discount factor?

Answer. $DF_{\text{USD}} = 1/1.019211 = 0.981151$. One dollar in 182 days is worth about 98.1 cents today. In the "foreign currency" picture: the exchange rate between "future dollars" and "today's dollars" is 0.9812.

2.5 Step 3 (L3 and L4): if she sold euros today

Question. If the exporter already had the euros and sold them spot, at which price would she deal, and how many dollars would she get? How much is one pip worth on this trade?

Answer. She **sells** EUR, the base currency. The bank buys from her at its **bid**, the lower number: 1.1580. She gets $8{,}000{,}000 \times 1.1580 = \text{USD } 9{,}264{,}000$.

A pip in EURUSD is 0.0001 USD per EUR. On EUR 8 million that is $0.0001 \times 8{,}000{,}000 = \text{USD } 800$ per pip. The 2-pip spread costs her one pip against mid (USD 800) to sell. A common slip: using the ask because "1.1582 is the price". Remember the money-changer: you always get the worse side.

2.6 Step 4 (L6): the fair forward price

But she does not have the euros yet. She will have them in 182 days. So she asks for a **6-month forward**.

Question. What is the fair forward mid, and how many forward points is that?

Answer. Two roads to the village "USD in 182 days" (L6). EUR is the base, USD is the quote currency:

$$F = 1.1581 \times \frac{1 + 0.038 \times 0.50556}{1 + 0.021 \times 0.50556} = 1.1581 \times \frac{1.019211}{1.010617} = 1.16795.$$

In words: the forward is spot, scaled up by how much dollars grow, and scaled down by how much euros grow.

Forward points: $(1.16795 - 1.1581) \times 10{,}000 \approx +98.5$ points. The euro is at a **forward premium** because it has the **lower** interest rate. Common sense: whoever holds euros for six months earns less interest, so the forward must give them more dollars per euro at the end to make the two roads cost the same.

Say the bank deals at 1.1679 (we ignore the bank's small forward spread to keep the arithmetic clean).

2.7 Step 5 (L5): what the hedge does at maturity

She **sells EUR 8 million forward at** $K = 1.1679$. That is a **short** EUR forward. Payoff in USD: $N(K - S_T) = 8{,}000{,}000 \times (1.1679 - S_T)$.

Spot in 182 days S_T	**Selling the EUR at spot**	**Forward payoff**	**Total USD**
1.1300 (euro fell)	9,040,000	+303,200	9,343,200
1.1679 (no change vs forward)	9,343,200	0	9,343,200
1.2000 (euro rose)	9,600,000	-256,800	9,343,200

The picture: the hedge is a see-saw. Whatever the euro does to her receivable, the forward does the opposite. The total is always $8{,}000{,}000 \times 1.1679 = \text{USD } 9{,}343{,}200$. She gave up the upside to remove the downside. In reality she would simply deliver the 8 million euros and receive USD 9,343,200; the table just shows why that is the same thing.

Note: she locked in USD 79,200 **more** than selling spot today (9,343,200 vs 9,264,000). That is not a gift. It is the interest difference between USD and EUR for six months, paid to her through the forward points.

2.8 Step 6 (L2 + L5): halfway there, what is the hedge worth?

Question. 91 days later the euro has weakened. The fair forward for the remaining 91 days is now 1.1500, and the USD rate is still 3.80%. What is her forward worth today?

Answer. If she closed it now, she would buy EUR 8 million forward at 1.1500 to offset her sale at 1.1679. The locked-in gain, paid in 91 days, is

$$8{,}000{,}000 \times (1.1679 - 1.1500) = \text{USD } 143{,}200.$$

That money arrives in 91 days, so discount it (L2): $DF = 1/(1 + 0.038 \times 91/360) = 0.990486$. Value today: $143{,}200 \times 0.990486 \approx \text{USD } 141{,}838$.

Where the picture stops working: this gain sits with the bank until maturity. If the bank failed, she would be an unsecured creditor for it. That is counterparty risk (L5), and it is why big hedges are often collateralised.

2.9 Likely interview questions

1. **"EUR rates are below USD rates. Is EURUSD forward above or below spot?"** Above. The lower-rate currency trades at a forward premium, so that no one can earn the rate gap for free.
2. **"USD 6-month rates rise by 50 basis points, everything else fixed. What happens to the forward points?"** They get bigger. With 4.30% the forward is about 1.17085, so points go from about +98.5 to about +127.5. Bigger rate gap, bigger premium.
3. **"A client sells you EUR 8 million at your bid, and before you hedge, EURUSD drops 10 pips. What did you lose?"** $10 \times 800 = \text{USD } 8{,}000$. This is the market maker's inventory risk from L3.

---

3. Algorithms & Data Structures: Fingerprints in a Coat Check

> **Where we are.** L1: Big-O counts steps. L2: C++ basics (`string`, `vector`, references). L3: scanning arrays and editing in place. L4: hash maps, the coat check with average $O(1)$ lookup. L5: prefix sums. L6: sorting, `std::sort` and comparators. Today's problem needs L1, L2, L4 and L6 together, plus a counting idea from the probability track.

3.1 The week in six pictures

Lesson	The picture to remember	The rule it gives
L1: Big-O	**Handshakes**: everyone shaking hands with everyone is n^2	For $n = 10^5$, $n^2 = 10^{10}$ is too slow; $n \log n \approx 1.7 \times 10^6$ is fine
L2: C++ refresher	A **reference is a nickname**, not a copy	Pass big things as `const T&`; copy only when you mean to
L3: arrays	An **odometer**: change the tail, keep the head	Scan once, edit in place, $O(1)$ extra space
L4: hash maps	A **coat check**: hand in a key, get your bucket back at once	`unordered_map`: insert and find in $O(1)$ on average
L5: prefix sums	A **running odometer of totals**	Sum of $a[l..r] = P_{r+1} - P_l$
L6: sorting	**Two clerks merging piles**	$O(n \log n)$; comparators must be strict (`<`, never `<=`)

3.2 The problem

A risk system receives a list of n short codes (lowercase words). Some codes are scrambles of each other, like `rates`, `stare` and `tears`. Two words are **anagrams** if they use exactly the same letters, the same number of times each.

a. Group the words so that anagrams are in the same group.

b. Count the **pairs** (i, j) with $i < j$ such that word i and word j are anagrams.

Constraints. $1 \le n \le 10^5$. Each word has length at most 20.

Example. Words: `rates, fx, stare, vol, bond, tears, xf, lov`.

Groups: `{bond}`, `{fx, xf}`, `{lov, vol}`, `{rates, stare, tears}`.

Pairs: the group of 3 gives 3 pairs, `fx/xf` gives 1, `vol/lov` gives 1, `bond` gives 0. Answer: **5**.

3.3 Brute force first

Check every pair of words. For each pair, decide "anagram or not" by sorting copies of both words and comparing them. There are $\binom{n}{2} \approx n^2/2$ pairs (L4 counting!), and each check costs $O(L \log L)$ for words of length L.

For $n = 10^5$ that is about 5×10^9 pair checks. Much too slow (L1).

3.4 The insight: give every word a fingerprint

The picture. Imagine a coat check where the ticket is not a random number but a **fingerprint** of the coat: its letters written in alphabetical order. `rates`, `stare` and `tears` all have the fingerprint `aerst`. `fx` and `xf` both become `fx`. Two words are anagrams **exactly when** their fingerprints match.

In the picture	In the code
The coat	A word
Its fingerprint	The word with its letters sorted (`std::sort` on a copy)
The coat check rack	`unordered_map<string, ...>` keyed by fingerprint
One hook on the rack	All words with one fingerprint: one anagram group

Now we never compare two words directly. Each word goes straight to its hook. That replaces the n^2 handshakes with n trips to the coat check.

Counting the pairs. If a hook ends up holding c words, it contributes $\binom{c}{2} = c(c-1)/2$ pairs (the bag of L4: a pair is a bag of 2). Check on the example: $\binom{3}{2} + \binom{2}{2} + \binom{2}{2} + \binom{1}{2} = 3 + 1 + 1 + 0 = 5$.

There is an even simpler way that needs only one pass: when a word arrives at a hook that already holds c words, it makes c new pairs, one with each of them. Add c, then increase the count. Over the whole run a hook of final size c adds $0 + 1 + 2 + \cdots + (c-1) = c(c-1)/2$, the same number. This "pair with everything already seen" pattern is the same one we used with prefix sums in L5.

Complexity. Each word: build the fingerprint by sorting L letters, $O(L \log L)$, then one hash map step, $O(L)$ on average to hash the string. Total $O(n \cdot L \log L)$ time, about $10^5 \times 20 \times 5 = 10^7$ basic steps. Space: the map stores at most n fingerprints, $O(nL)$.

(A faster fingerprint: count each of the 26 letters and use the 26 counts as the key. That is $O(L)$ per word. With $L \le 20$ the sorted string is simpler and just as fast in practice.)

Watch the overflow. If all 10^5 words are the same, the answer is $\binom{10^5}{2} \approx 5 \times 10^9$, which does not fit in a 32-bit `int` (max about 2.1×10^9). Use `long long`.

3.5 C++ solution

```cpp
// Daily Quant Digest 2026-11-01 (Revision day): hash maps + sorting + counting.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>
```

```cpp
// The "fingerprint" of a word: its letters in sorted order.
std::string fingerprint(const std::string& w) {
    std::string key = w;                            // copy, then sort the copy
    std::sort(key.begin(), key.end());
    return key;
}

// (a) Group the words by fingerprint, in a predictable order.
std::vector<std::vector<std::string>> groupAnagrams(const std::vector<std::string>& words) {
    std::unordered_map<std::string, std::vector<std::string>> buckets;
    for (const std::string& w : words)
        buckets[fingerprint(w)].push_back(w);       // [] creates an empty bucket if new

    std::vector<std::vector<std::string>> groups;
    groups.reserve(buckets.size());
    for (auto& [key, group] : buckets) {            // structured binding (C++17)
        std::sort(group.begin(), group.end());
        groups.push_back(std::move(group));         // move: no copy of the strings
    }
    std::sort(groups.begin(), groups.end(),
              [](const std::vector<std::string>& x, const std::vector<std::string>& y) {
                  return x.front() < y.front();     // strict "<", never "<="
              });
    return groups;
}

// (b) Count anagram pairs in one pass.
long long countPairs(const std::vector<std::string>& words) {
    std::unordered_map<std::string, long long> seen;
    long long pairs = 0;
    for (const std::string& w : words) {
        long long& c = seen[fingerprint(w)];        // reference into the map
        pairs += c;                                 // pair with each earlier match
        ++c;                                        // then count this word too
    }
    return pairs;
}

int main() {
    std::vector<std::string> ex = {"rates", "fx", "stare", "vol", "bond", "tears", "xf", "lov"};
    for (const auto& g : groupAnagrams(ex)) {
        std::cout << "{";
        for (std::size_t i = 0; i < g.size(); ++i) std::cout << (i ? ", " : "") << g[i];
        std::cout << "}\n";
    }
    std::cout << "pairs = " << countPairs(ex) << "\n";   // prints 5
    assert(countPairs(ex) == 5);
    return 0;
}
```

The full file `2026-11-01.cpp` also has the brute force, a randomized cross-check (20,000 random lists, with the extra check that the group sizes add up to n and that $\sum \binom{c}{2}$ equals the one-pass count), and a test with 100,000 copies of one word (answer 4,999,950,000, which needs `long long`). Output:

```
{bond}
{fx, xf}
{lov, vol}
{rates, stare, tears}
pairs = 5
```

3.6 Walkthrough of the C++

`fingerprint(const std::string& w).` The parameter is a `const` reference (L2): a nickname for the caller's string, no copy, and the promise not to change it. Inside, `std::string key = w;` makes the one copy we actually want, because we are about to sort it. `std::sort(key.begin(), key.end())` sorts the characters: a `std::string` is a sequence of `char`, and `begin()`/`end()` are iterators to its first character and one past its last, exactly like a `vector`. Characters compare by their codes, and `'a' < 'b' < ... < 'z'`, so the result is alphabetical.

`std::unordered_map<std::string, std::vector<std::string>> buckets;` The coat check rack (L4). The key is the fingerprint; the value is the list of words on that hook.

`buckets[fingerprint(w)].push_back(w);` The square brackets do two jobs. If the fingerprint is already a key, they return a reference to its vector. If not, they first **create** an empty vector for it, then return a reference to that. So one line handles "new group" and "existing group" alike. (`find` or `at` would not create anything; here creating is exactly what we want.)

`for (auto& [key, group] : buckets).` Each element of an `unordered_map` is a `std::pair<const Key, Value>`. The C++17 **structured binding** `[key, group]` gives the two halves names, like unpacking a tuple in Python. The `&` makes them references into the map, so `std::sort(group...)` sorts the vector stored inside the map, not a copy. The key is `const`, so you could not change it even if you tried: changing a key would put the coat on the wrong hook.

`groups.reserve(buckets.size());` Tells the vector how many elements are coming so it allocates memory once instead of growing step by step. Not required, just tidy.

`groups.push_back(std::move(group));` `std::move` says "I am done with `group`, you may take its insides". The vector's internal buffer is handed over instead of copying every string. After this, `group` is left empty but valid. This is safe here because we never use the map's vectors again.

The lambda comparator. `[](const auto& x, const auto& y) { return x.front() < y.front(); }` is a small unnamed function (L6). The empty `[]` means it captures nothing from the surrounding code. It sorts groups by their first word, which is the smallest word in the group because we sorted each group first. It uses strict `<`: a comparator must answer "must x come **before** y?", and with `<=` two equal items would each claim to come first, which can break `std::sort`. (Two groups can never share a first word anyway: that word would have one fingerprint.)

Why sort the output at all? An `unordered_map` stores its keys in no particular order, and that order can differ between compilers. Sorting makes the output the same everywhere, which is what an interviewer (or a test) expects.

`long long& c = seen[fingerprint(w)];` The key line of part (b). `seen[...]` creates the counter at 0 if the fingerprint is new (a `long long` value-initialises to 0). The `&` makes `c` a **reference to the number stored inside the map**, so `++c` updates the map itself. Without the `&`, `c` would be a copy and the map would never change: every count would stay 0 and the answer would be 0. This is the most common bug in this pattern.

`pairs += c; ++c;` The order matters. First pair the new word with the c words already on the hook, then add the new word. Swapping the two lines would pair each word with itself and give n too many.

Testing pattern in `main`. As in earlier days: worked examples with `assert`, then a seeded `std::mt19937` random generator to cross-check the fast answer against the brute force on thousands of small inputs. A tiny alphabet (`a`, `b`, `c`) and short words make collisions common, so the anagram logic is really exercised.

3.7 Self-check (answers below)

1. What does `countPairs` return for `{"ab", "ba", "ab", "ba"}`?
2. Why does `fingerprint` take `const std::string&` but then copy it, instead of taking `std::string` by value?
3. If the words could contain uppercase letters and you wanted `Rates` and `stare` to match, what one change would you make?

Answers. (1) All four share the fingerprint `ab`, so $\binom{4}{2} = 6$. (2) Either works. Taking it by value would also make exactly one copy (and is a common idiom); the `const&` plus explicit copy makes the copy visible to the reader. What you must not do is take a non-const reference and sort it, which would scramble the caller's word. (3) Lower-case every character before sorting, e.g. with `std::tolower` inside a loop over `key`.

---

4. Music: The Kinks, "Waterloo Sunset" (1967)

Context

"Waterloo Sunset" was written by **Ray Davies**, the Kinks' singer and songwriter, and released as a single in May 1967. It reached number 2 in the UK. It also appears on the album *Something Else by the Kinks*, released later that year. The band was Ray Davies, his younger brother **Dave Davies** on lead guitar, **Pete Quaife** on bass and **Mick Avory** on drums. It was never a big hit in the United States, which tells you something about how English it is.

The story behind it

The song is a view of London from the Thames: the river, Waterloo station, crowds of people hurrying home. The narrator stays at his window watching the sunset and feels safe and content on his own, while a young couple named Terry and Julie meet at the station

and cross the river together.

Davies grew up in north London and has said the river meant a lot to him from childhood. He has also said the song started out as “Liverpool Sunset” before he moved it to London. For years people guessed that Terry and Julie were the actors Terence Stamp and Julie Christie, who were famous at the time; Davies has said that was not who he had in mind when he wrote it.

Why people love it

It is three minutes of calm in a loud decade. 1967 was the year of psychedelia and big studio experiments, and here is a quiet song about a man looking out of a window. Many critics and musicians call it one of the most beautiful pop songs ever written. It feels like a short film: you can see the river turning gold. And the feeling is gentle and a little sad at the same time, like watching a city you love from a distance.

What to listen for

- **The opening.** The song opens with a gently falling line. That downward motion sets the mood of a sun going down before a word is sung.
- **The backing vocals.** High, soft harmonies float behind Ray. They were sung by Dave Davies, Pete Quaife and Ray’s wife at the time, Rasa. Notice how they make the song sound wider and more dreamy.
- **Ray’s voice.** He sings plainly, almost conversationally, with no showing off. That plainness is why the song feels honest.
- **The arrangement.** It is a small, uncluttered record built around the band itself. Notice how much atmosphere they make with so little.
- **The contrast.** The verses describe the busy, slightly cold city; the chorus returns to the narrator’s peaceful view. Listen to how the music lifts each time it comes back to the sunset.